

DataCORE

Advanced Pandas, Publication-Quality Figures & Tables

Afternoon: DataCORE

1. **Advanced Pandas — GroupBy, Pivot Tables, Merging** (1:00–2:00)
2. **Publication-Quality Figures** (2:00–2:45)
3. **Scientific Storytelling — Table Captions** (2:45–3:00)

*This morning you met your data. This afternoon you **slice it by groups**, build **publication-ready tables and figures**, and learn to **caption** them like a journal article.*

1:00 – 2:00 · Advanced Pandas

From one variable to **many**

- So far: one variable at a time (a mean, a histogram).
- Real analysis asks **comparative** questions: *does the pattern differ **by group**?*
- Three workhorse tools do almost all of it:

 **groupby** — summaries within each group

 **pivot_table** — publication-style tables

 **merge** — combine two datasets on a key

Our data: the same wage survey — education, gender, region, experience_years, hourly_wage.

Load the data

Python R

```
wages = pd.read_csv("wages.csv")
print(wages.head())
```

	education	gender	region	experience_years	hourly_wage
0	Bachelor	Male	Rural	10	28.54
1	Associate	Male	Urban	11	28.03
2	Graduate	Female	Suburban	4	31.56
3	Bachelor	Female	Urban	16	24.90
4	High School	Male	Urban	16	27.83

```
print("N =", len(wages))
```

N = 220

 One row per worker. We'll summarise **across** workers, grouped by their characteristics.

GroupBy – multi-level summaries

Python R

```
print(wages.groupby(["education", "gender"])["hourly_wage"]    # group by TWO keys
      .agg(["mean", "std", "count"]).round(2))
```

		mean	std	count
education	gender			
Associate	Female	22.80	3.74	19
	Male	25.58	5.58	26
Bachelor	Female	28.05	4.20	40
	Male	30.42	3.57	40
Graduate	Female	33.38	4.33	17
	Male	36.45	2.81	10
High School	Female	20.64	4.19	35
	Male	24.09	4.01	33

 Grouping by **two** keys gives one row per *combination*. `agg(...)` computes several statistics at once — mean, spread, and sample size together.

Pivot table – a publication-style layout

Python R

```
print(pd.pivot_table(wages, values="hourly_wage",  
                     index="education", columns="gender",  
                     aggfunc="mean").round(2))
```

gender	Female	Male
education		
Associate	22.80	25.58
Bachelor	28.05	30.42
Graduate	33.38	36.45
High School	20.64	24.09

 A pivot turns **groups into rows and columns** — the exact shape of a results table you'd drop into a paper or slide.

Merge – combine two datasets on a key

Python R

```
region_info = pd.read_csv("region_info.csv")           # cost of living by region
merged = pd.merge(wages, region_info, on="region")      # join on the shared key
print(merged.head())
```

	education	gender	region	experience_years	hourly_wage	cost_of_living_index	median_rent
0	Bachelor	Male	Rural	10	28.54	88	950
1	Associate	Male	Urban	11	28.03	121	1980
2	Graduate	Female	Suburban	4	31.56	104	1450
3	Bachelor	Female	Urban	16	24.90	121	1980
4	High School	Male	Urban	16	27.83	121	1980

 Merging matches rows by a **shared key** (here `region`) – the same way you'd join county health data to county income data on a **FIPS code**.

Team Task

Which groupby matters for **your** question? – ~12 min

1. Pick the **grouping variable** most relevant to your research question.
2. Run `df.groupby('group')`
`['outcome'].agg(['mean', 'std', 'count'])` (Python) **or**
`the group_by() |> summarise()` (R).
3. **Stretch:** group by **two** variables, or build a `pivot_table`.

Then answer in one sentence:

*"Grouped by **[variable]**, the **[outcome]** was highest for **[group]** (mean = __) and lowest for **[group]** (mean = __)."*

What does that tell you about your question?

12:00

2:00 – 2:45 · Publication-Quality Figures

One rule

 **If you would be embarrassed to show this chart in a conference presentation, do not put it in your slides.**

Readable

Clear axis labels with units, a title, legible font sizes.

Honest

Show uncertainty (error bars); don't truncate axes to exaggerate.

Accessible

Colorblind-safe palettes; don't rely on red/green alone.

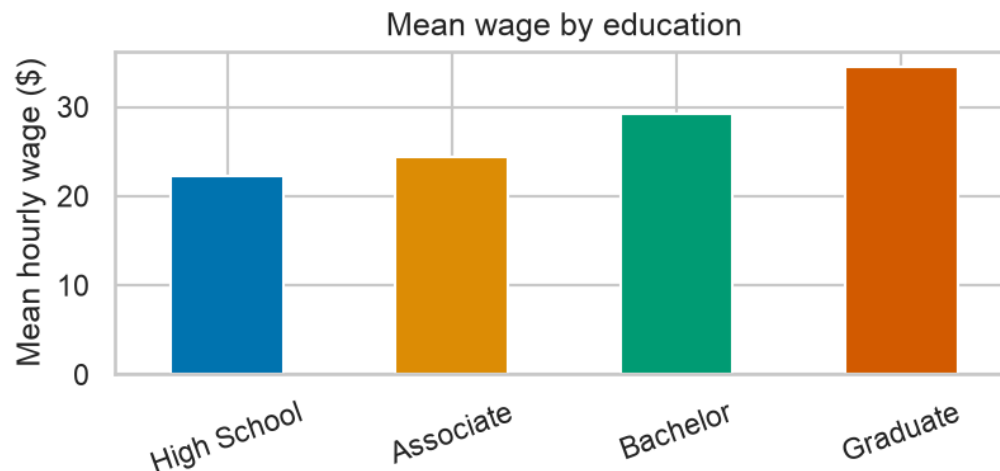
 We'll upgrade a plain chart step by step: **palette** → **error bars** → **annotation** → **export at 300 DPI**.

Colorblind-safe palettes

Python (seaborn)

R (ggplot2)

```
order = wages.groupby("education")["hourly_wage"].mean().sort_values()
colors = sns.color_palette("colorblind", len(order)) # colorblind-safe
order.plot(kind="bar", color=colors)
plt.ylabel("Mean hourly wage ($)"); plt.xlabel("")
plt.title("Mean wage by education")
plt.xticks(rotation=20); plt.tight_layout(); plt.show()
```

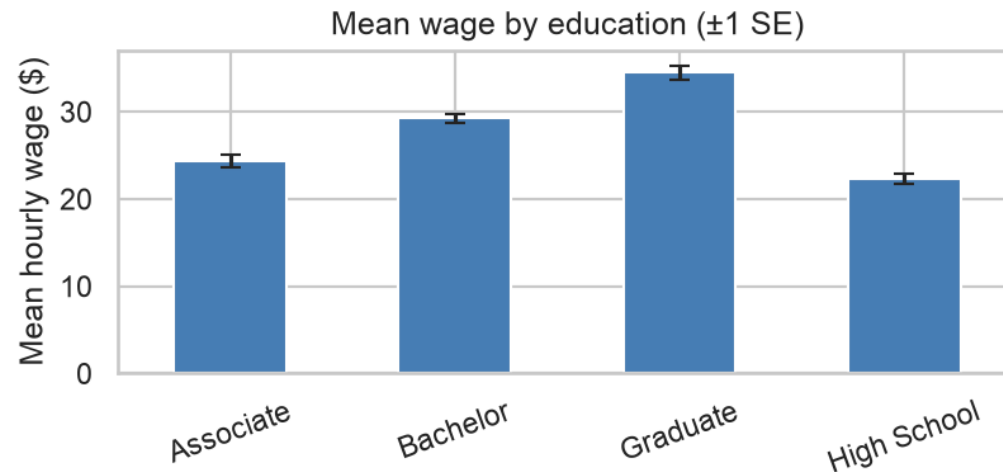


seaborn's **"colorblind"** palette and ggplot's **viridis** are designed to stay distinguishable for color-vision deficiencies.

|| Error bars – show the uncertainty

Python R

```
g = wages.groupby("education")["hourly_wage"]
means = g.mean(); ses = g.std() / np.sqrt(g.count()) # standard error
means.plot(kind="bar", yerr=ses, capsize=4, color="#4a7fb5")
plt.ylabel("Mean hourly wage ($)"); plt.xlabel("")
plt.title("Mean wage by education ( $\pm 1$  SE)")
plt.xticks(rotation=20); plt.tight_layout(); plt.show()
```



i A bar without error bars hides how **certain** the average is. **mean ± 1 SE** is the standard add-on.

📍 Annotate the point that matters

Python R

```
x = wages["experience_years"]; y = wages["hourly_wage"]
plt.scatter(x, y, alpha=0.5, color="#4a7fb5")
i = y.idxmax() # highest earner
plt.annotate("highest earner", (x[i], y[i]), xytext=(x[i]-13, y[i]+1),
          arrowprops=dict(arrowstyle="->", color="#D55E00"))
plt.xlabel("Years of experience"); plt.ylabel("Hourly wage ($)")
plt.tight_layout(); plt.show()
```



💬 One well-placed label guides the eye to your **key data point** — far better than making the viewer hunt for it.

Export at 300 DPI for slides & print

Python R

```
plt.savefig("fig1_wage_by_education.png",  
            dpi=300, bbox_inches="tight")    # crisp; no clipped labels
```

 Give every figure a **descriptive file name** (fig1_wage_by_education.png, not Untitled.png) and a **caption**. `bbox_inches="tight"` stops axis labels from being cut off.

Team Task

Your 2–3 best charts – ~12 min

1. Choose the **2–3 charts** that best answer your research question.
2. Upgrade each: **colorblind-safe palette**, clear labels with **units**, a title.
3. Add **error bars** or **annotate** the key point where it helps.
4. **Save** each at **300 DPI** with a descriptive file name.

Each chart needs:

- A descriptive **file name**
- A one-line **caption**
- A reason it's in the deck – *what question does it answer?*

12:00

2:45 – 3:00 · Scientific Storytelling

A table is useless without a caption

Python R

```
print(wages.groupby("education")["hourly_wage"]  
      .agg(["mean", "median", "std", "min", "max"]).round(2))
```

	mean	median	std	min	max
education					
Associate	24.41	23.53	5.04	12.13	34.17
Bachelor	29.24	29.10	4.05	19.06	37.72
Graduate	34.51	34.98	4.07	26.16	42.03
High School	22.32	22.22	4.43	10.96	31.28

❓ A reader who sees only this table should still know **what it shows, where it came from, and one thing to notice**. That's the caption's job.

APA table format

“ **Table 1.** *Hourly wages by education level (N = 220 workers).* **Note.** Data from the 2023 wage survey; values are in U.S. dollars per hour. Mean hourly wage rises with education, from \\$22 (high school) to \\$35 (graduate). ”

The shape:

"Table 1. [Descriptive title]. Note. [Source and any key definitions]."

Why it matters:

APA table format is the **standard** in the social and health sciences — titles above the table, notes below.

Write your table's caption – now

Write a caption for **your** summary-statistics table:

*"Table 1. **[descriptive title]**. Note. **[source, sample, definitions]**; **[one notable pattern]**."*

Read it to a partner who hasn't seen your data.

✓ Checklist:

- Descriptive **title** (not just "Table 1")?
- **Sample / N** stated?
- **Source & year**?
- Units / key **definitions**?
- **One pattern** worth noticing?

10:00

